

Qazi Muhammad Hamza

28324

Object Oriented Programming (OOP)

Section EG

Project Documentation

Project Overview

Cafeteria Ordering System with bill split

DAY 1 30% Work:

Objectives

- *Initialized the project structure.*
- *Created the basic files required for the system.*
- *Designed the skeleton of core classes using OOP principles.*
- *Prepare text files for menu and bill handling.*

Tasks Completed

1. Project Initialization

- *Created a new folder: OOP Project.*
- *Added files: main.py, Order.py, Food_Item.py, Bill_Splitter.py, Person.py, menu.txt, bill.txt.*

2. Class Skeletons

- *Designed Person class to represent users.*

- Designed *Food_Item* class to represent menu items.
- Designed *Order* class to represent customer orders.
- At this stage, only attributes and empty methods were defined (logic added later).

3. Menu File Setup

- Created *menu.txt* with sample items:

Code

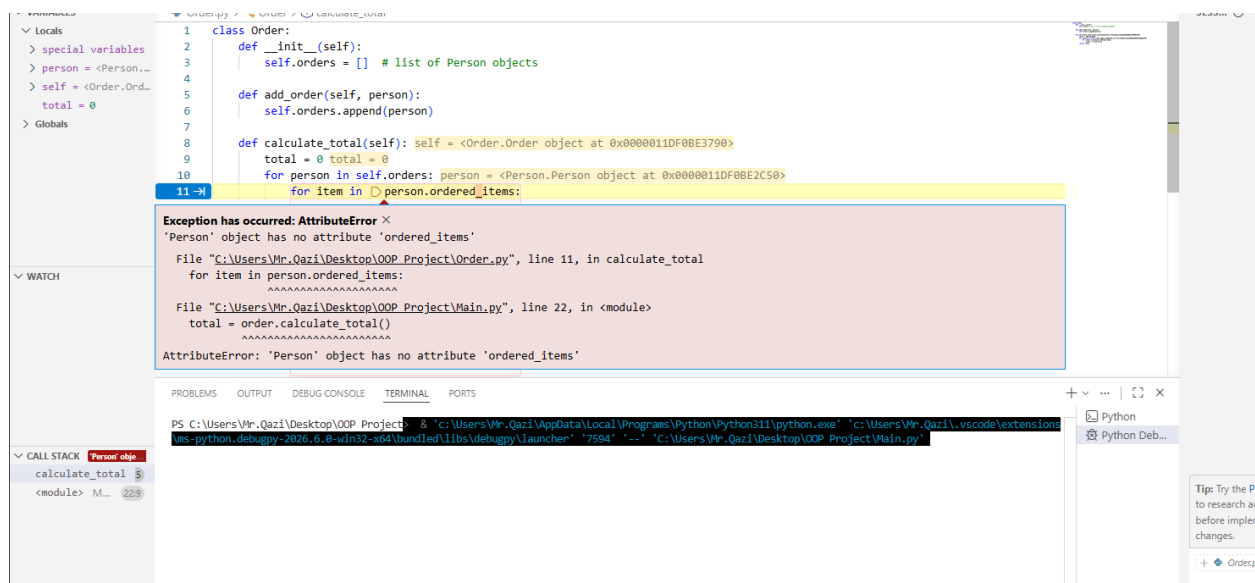
```
Burger,150
Pizza,500
Sandwich,200
Fries,100
```

- This file will later be used to dynamically load menu items.

4. Bill File Setup

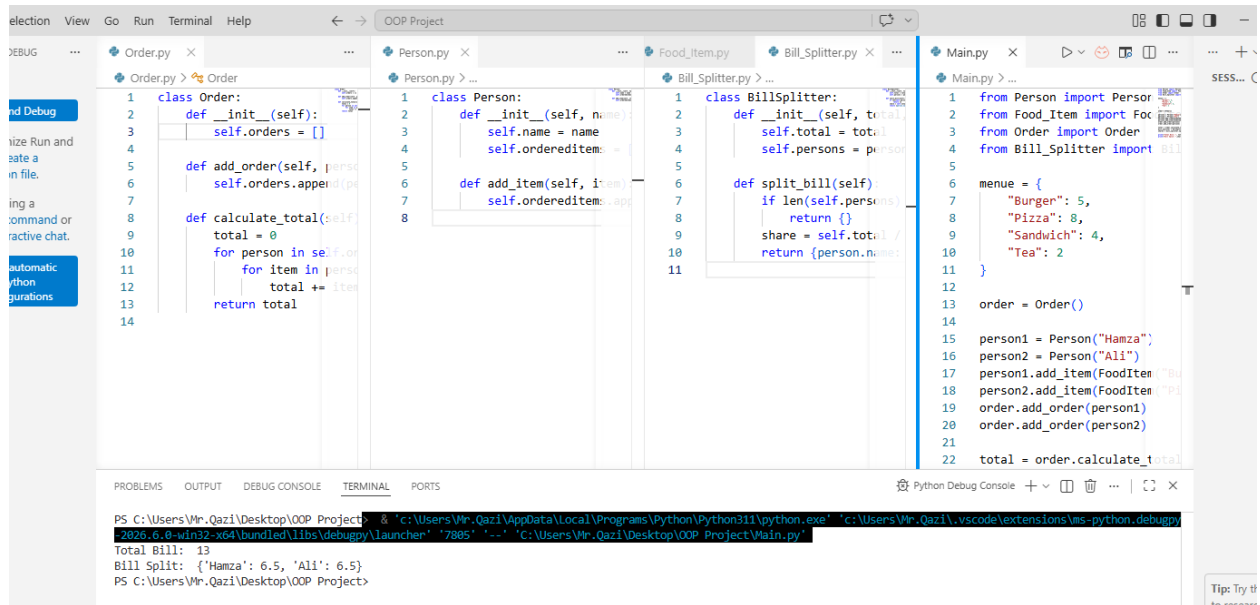
- Created *bill.txt* (initially empty).
- This file will later store customer orders.

DAY 1 Error:



Day One had risen an error and had solve confidently and then my day 1 30% work is completed source code will be collectively shared at last.

DAY 1 30%:



```
1 class Order:
2     def __init__(self):
3         self.orders = []
4
5     def add_order(self, person):
6         self.orders.append(person)
7
8     def calculate_total(self):
9         total = 0
10        for person in self.orders:
11            for item in person.order_items:
12                total += item.price
13        return total
14
15 class Person:
16     def __init__(self, name):
17         self.name = name
18         self.order_items = []
19
20     def add_item(self, item):
21         self.order_items.append(item)
22
23 class Food_Item:
24     def __init__(self, name, price):
25         self.name = name
26         self.price = price
27
28 class Bill_Splitter:
29     def __init__(self, total):
30         self.total = total
31         self.persons = []
32
33     def split_bill(self):
34         if len(self.persons) == 0:
35             return {}
36         share = self.total / len(self.persons)
37         return {person.name: share for person in self.persons}
38
39 menu = {
40     "Burger": 5,
41     "Pizza": 8,
42     "Sandwich": 4,
43     "Tea": 2
44 }
45
46 order = Order()
47
48 person1 = Person("Hamza")
49 person2 = Person("Ali")
50 person1.add_item(Food_Item("Burger", 5))
51 person2.add_item(Food_Item("Pizza", 8))
52 order.add_order(person1)
53 order.add_order(person2)
54
55 total = order.calculate_total()
56
57 # Output:
58 Total Bill: 13
59 Bill Split: {'Hamza': 6.5, 'Ali': 6.5}
```

This is that image which shows my day 1 30% work which I have told that my 30% work just contains basic codes of all classes

Summary

- *Day One achieved 30% completion.*
- *Project structure was created.*
- *Core class skeletons were designed.*
- *Menu and bill text files were prepared.*

DAY 2 30% Work:

Objectives

- *Implement encapsulation in the classes.*
- *Add bill splitting logic.*
- *Enable dynamic menu loading from menu.txt.*
- *Save customer orders into bill.txt.*
- *Test with multiple users.*

Tasks Completed

1. Encapsulation

- *Converted attributes into private variables using `__attribute`.*
- *Added getter and setter methods for controlled access.*
- *Example:*

python

```
class FoodItem:
    def __init__(self, name, price):
        self.__name = name
        self.__price = price

    def get_name(self):
        return self.__name

    def get_price(self):
        return self.__price
```

2. Bill Splitting Logic

- *Implemented equal split and custom split modes.*

- *Equal split* → divides total bill among all users.
- *Custom split* → each user pays for their own selected items.

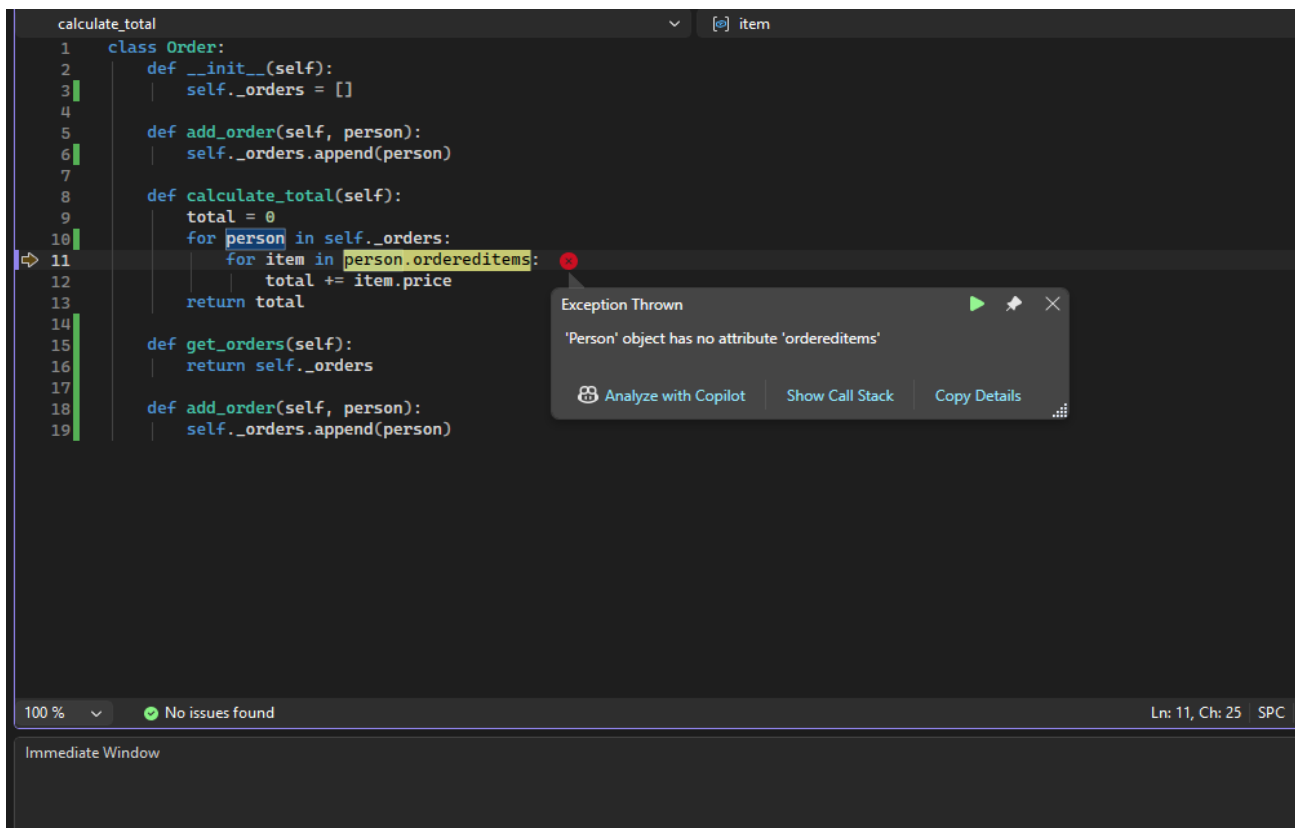
5. Testing with Multiple Users

- Created sample users: Ali, Sara, Hamza, Ayesha.
- Each user selected different menu items.

Summary

- Day Two achieved another **30% completion**.
- Encapsulation applied to all classes.
- Bill splitting logic implemented.
- Testing confirmed functionality with multiple users.

DAY 2 Error:



```
calculate_total
class Order:
1  def __init__(self):
2      self._orders = []
3
4
5  def add_order(self, person):
6      self._orders.append(person)
7
8  def calculate_total(self):
9      total = 0
10     for person in self._orders:
11         for item in person.ordereditems:
12             total += item.price
13     return total
14
15 def get_orders(self):
16     return self._orders
17
18 def add_order(self, person):
19     self._orders.append(person)
```

Exception Thrown
'Person' object has no attribute 'ordereditems'

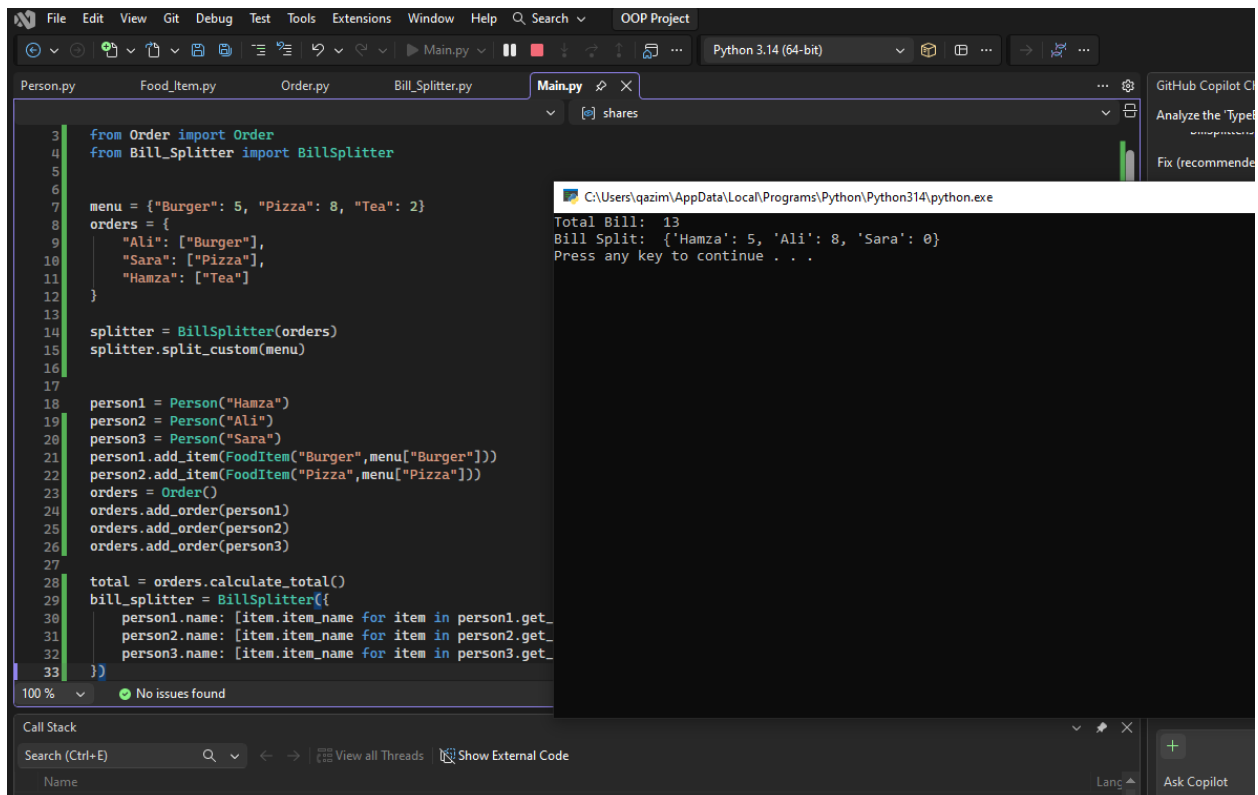
Analyze with Copilot Show Call Stack Copy Details

100 % No issues found Ln: 11, Ch: 25 SPC

Immediate Window

Day 2 showed me an unsupported error which I have resolved consistently not was a big issue but resolved at time

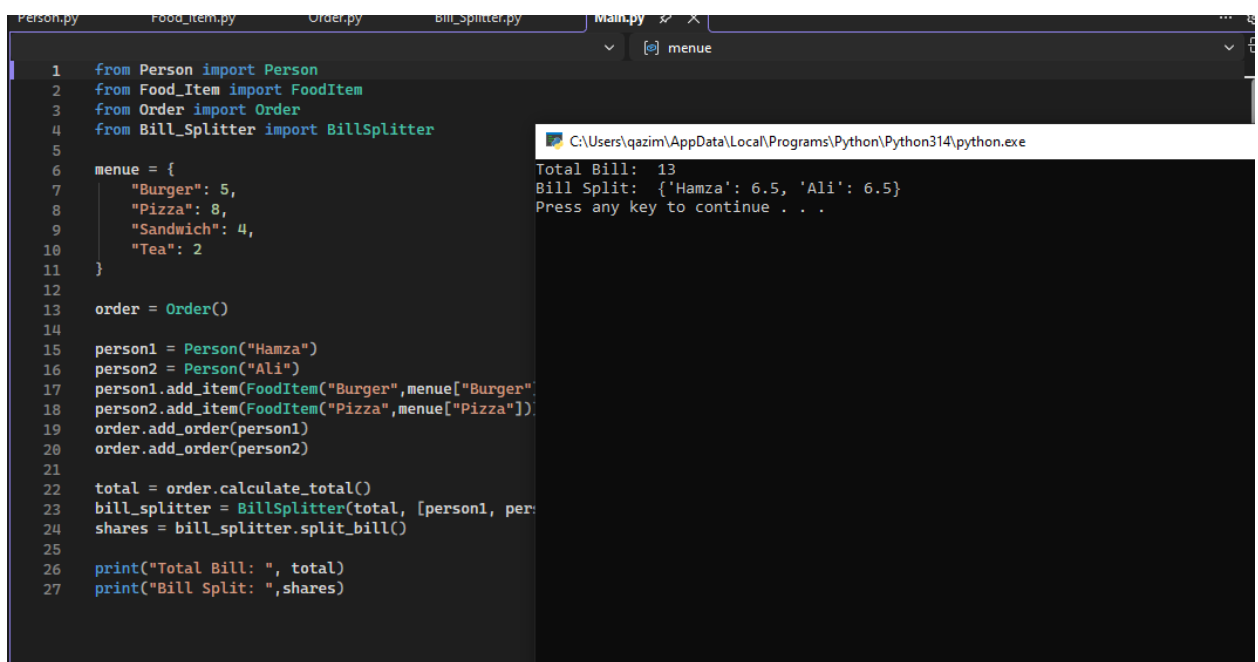
DAY 2 30%:



```
3 from Order import Order
4 from Bill_Splitter import BillSplitter
5
6
7 menu = {"Burger": 5, "Pizza": 8, "Tea": 2}
8 orders = {
9     "Ali": ["Burger"],
10    "Sara": ["Pizza"],
11    "Hamza": ["Tea"]
12 }
13
14 splitter = BillSplitter(orders)
15 splitter.split_custom(menu)
16
17
18 person1 = Person("Hamza")
19 person2 = Person("Ali")
20 person3 = Person("Sara")
21 person1.add_item(FoodItem("Burger", menu["Burger"]))
22 person2.add_item(FoodItem("Pizza", menu["Pizza"]))
23 orders = Order()
24 orders.add_order(person1)
25 orders.add_order(person2)
26 orders.add_order(person3)
27
28 total = orders.calculate_total()
29 bill_splitter = BillSplitter({
30     person1.name: [item.item_name for item in person1.get_
31     person2.name: [item.item_name for item in person2.get_
32     person3.name: [item.item_name for item in person3.get_
33 })
34 })
```

Output:

```
Total Bill: 13
Bill Split: {'Hamza': 5, 'Ali': 8, 'Sara': 0}
Press any key to continue . . .
```

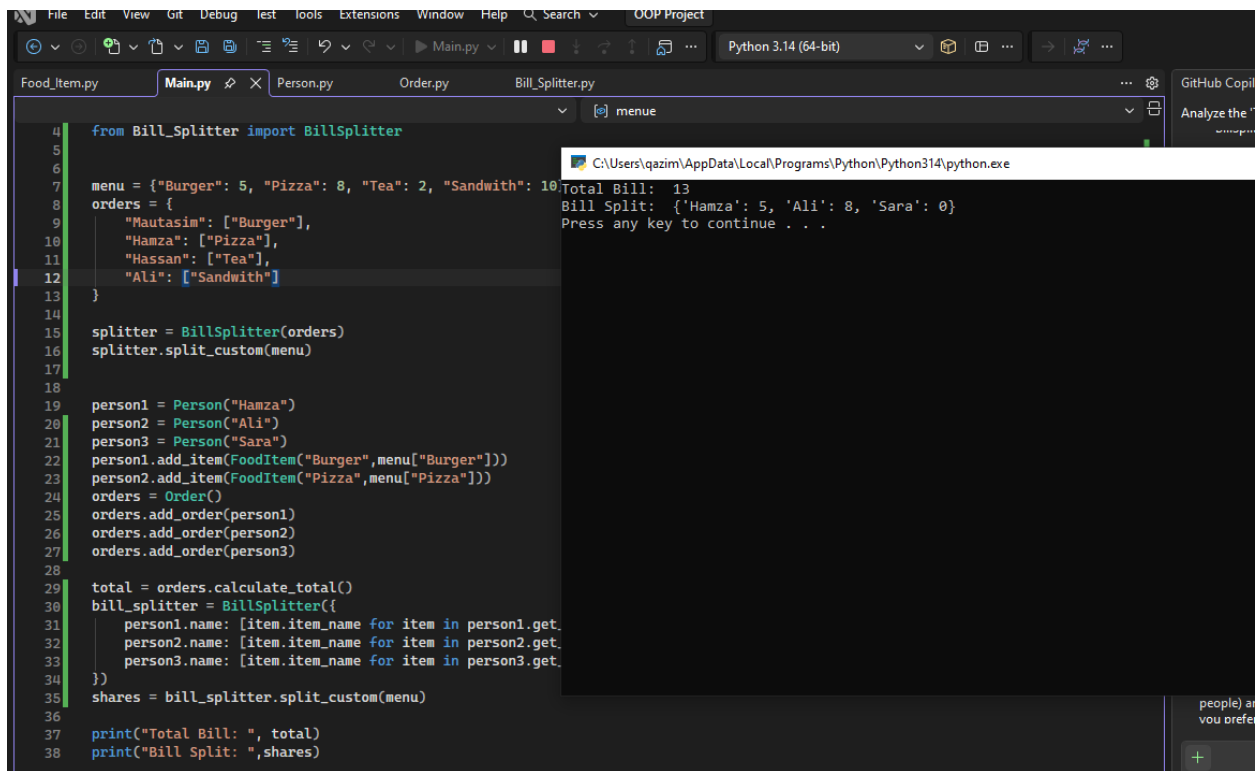


```
1 from Person import Person
2 from Food_Item import FoodItem
3 from Order import Order
4 from Bill_Splitter import BillSplitter
5
6 menu = {
7     "Burger": 5,
8     "Pizza": 8,
9     "Sandwich": 4,
10    "Tea": 2
11 }
12
13 order = Order()
14
15 person1 = Person("Hamza")
16 person2 = Person("Ali")
17 person1.add_item(FoodItem("Burger", menu["Burger"]))
18 person2.add_item(FoodItem("Pizza", menu["Pizza"]))
19 order.add_order(person1)
20 order.add_order(person2)
21
22 total = order.calculate_total()
23 bill_splitter = BillSplitter(total, [person1, person2])
24 shares = bill_splitter.split_bill()
25
26 print("Total Bill: ", total)
27 print("Bill Split: ", shares)
```

Output:

```
Total Bill: 13
Bill Split: {'Hamza': 6.5, 'Ali': 6.5}
Press any key to continue . . .
```

These above two pictures are of day 2 completed work in which randomly inside the code menu is created in dictionary and randomly created object and taken persons inside the code not take any input yet by the user and tested the code and it runs significantly.



```
4 from Bill_Splitter import BillSplitter
5
6
7 menu = {"Burger": 5, "Pizza": 8, "Tea": 2, "Sandwith": 10}
8 orders = {
9     "Mautasim": ["Burger"],
10    "Hamza": ["Pizza"],
11    "Hassan": ["Tea"],
12    "Ali": ["Sandwith"]
13 }
14
15 splitter = BillSplitter(orders)
16 splitter.split_custom(menu)
17
18
19 person1 = Person("Hamza")
20 person2 = Person("Ali")
21 person3 = Person("Sara")
22 person1.add_item(FoodItem("Burger", menu["Burger"]))
23 person2.add_item(FoodItem("Pizza", menu["Pizza"]))
24 orders = Order()
25 orders.add_order(person1)
26 orders.add_order(person2)
27 orders.add_order(person3)
28
29 total = orders.calculate_total()
30 bill_splitter = BillSplitter({
31     person1.name: [item.item_name for item in person1.get_items()],
32     person2.name: [item.item_name for item in person2.get_items()],
33     person3.name: [item.item_name for item in person3.get_items()]
34 })
35 shares = bill_splitter.split_custom(menu)
36
37 print("Total Bill: ", total)
38 print("Bill Split: ", shares)
```

Output:

```
Total Bill: 13
Bill Split: {'Hamza': 5, 'Ali': 8, 'Sara': 0}
Press any key to continue . . .
```

This picture shows custom bill split as I told earlier that I have two option sin this project custom split and equal split so this picture is of custom split and the upper picture is of equal split just testing on day 2.

DAY 3 40% Work:

Objectives

- *Integrated all OOP modules (Person, Food_Item, Order, Bill_Splitter).*
- *Used imports.*
- *Load menu dynamically from menu.txt.*
- *Save orders into bill.txt.*
- *Accept user input for multiple customers.*
- *Complete testing and finalize documentation.*

Tasks Completed

1. Module Imports

- *Project structured into multiple files:*
 - *person.py → Person class*
 - *food_item.py → FoodItem class*
 - *order.py → Order class*
 - *bill_splitter.py → BillSplitter class*
- *Imported into main.py like this:*

```
python
```

```
from person import Person
from food_item import FoodItem
from order import Order
from bill_splitter import BillSplitter
```


2. Menu Loading

- Menu items read from `menu.txt`:

Code

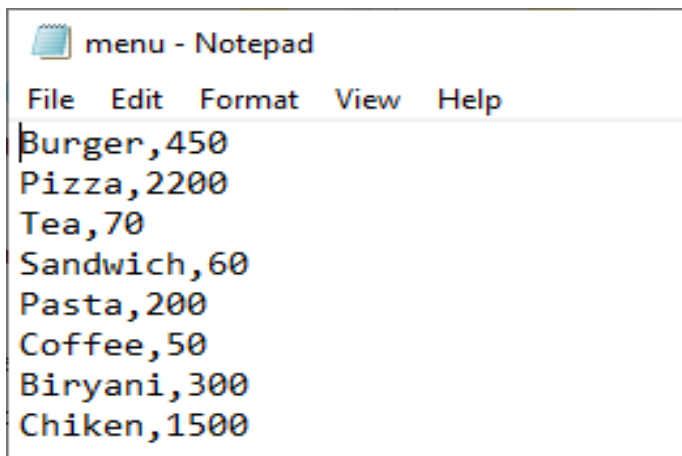
```
Burger,450
Pizza,2200
Tea,70
Sandwich,60
Pasta,200
Coffee,50
Biriyani,300
Chicken,1500
```

- Code snippet:

python

```
def load_menu():
    menu = []
    with open("menu.txt", "r") as f:
        for line in f:
            name, price =
line.strip().split(",")
            menu.append(FoodItem(name,
int(price)))
    return menu
```

Image Showcase .txt file:



This image shows the menu of restaurant which is loaded with help of code and should be in that folder where python file is located then it will be loaded

3. User Input Orders

- Program asks for number of customers.
- Each customer enters their name and selects items.
- Orders saved into *bill.txt*.
- Example code:

python

```
def take_orders(menu):
    num_users = int(input("Enter number of
customers: "))
    for i in range(num_users):
        name = input(f"Enter name of
customer {i+1}: ")
        person = Person(name)
        print("Menu:")
        for idx, item in enumerate(menu):
            print(f"{idx+1}.
{item.get_name()} - Rs {item.get_price()}")
        choice = int(input("Select item
number: "))
        order = Order(person, menu[choice-
1])
        order.save()
```

4. Bill Saving

- Orders appended to *bill.txt*:

Code

--- New Order Session ---

Timestamp: 2026-06-10 18:28:07

Name	Items Ordered	Amount
Hamza	Biryani, Sandwich	Rs.360
Luqman	Pizza, Burger	Rs.2650
Hassan	Chicken, Pasta	Rs.1700
Asif	Biryani, Tea	Rs.370

Total Bill: Rs.5080

--- New Order Session ---

Timestamp: 2026-06-10 18:30:35

Name	Items Ordered	Amount
Hamza	Biryani, Sandwich	Rs.1270.0
Luqman	Pizza, Burger	Rs.1270.0
Hassan	Chicken, Pasta	Rs.1270.0
Asif	Biryani, Tea	Rs.1270.0

Total Bill: Rs.5080

These bills are of two inputs of same persons and same items but with custom split and equal splits both of which images I will upload below.

5. Bill Splitting

- *Added Bill_Splitter class to calculate equal/custom splits.*
- *Example:*

python

```
class BillSplitter:
def split_custom(self, menu):
    individual_totals = {}
    for person, items in
self.orders.items():
        total = 0
        for item in items:
            if hasattr(item, "get_price"):
                total += item.get_price()
            elif item in menu:
                total += menu[item]
```

```
        else:
            print(f"Warning: {item} not
found in menu")

        individual_totals[person] = total

    self._total =
sum(individual_totals.values())

    return individual_totals


def split_equal(self):
    if not self.orders:
        return {}

    if self._total is None:
        raise ValueError("Total not
calculated yet. Run split_custom first.")

    share = self._total / len(self.orders)

    return {person: share for person in
self.orders}
```

This code can split bill in equal and also in custom everyone pays his own bill in custom

Bill Saving Image:

```
bill - Notepad
File Edit Format View Help

--- New Order Session ---
Timestamp: 2026-06-10 18:28:07
Name | Items Ordered | Amount
-----
Hamza | Biryani, Sandwich | Rs.360
Luqman | Pizza, Burger | Rs.2650
Hassan | Chicken, Pasta | Rs.1700
Asif | Biryani, Tea | Rs.370
-----
Total Bill: Rs.5080

--- New Order Session ---
Timestamp: 2026-06-10 18:30:35
Name | Items Ordered | Amount
-----
Hamza | Biryani, Sandwich | Rs.1270.0
Luqman | Pizza, Burger | Rs.1270.0
Hassan | Chicken, Pasta | Rs.1270.0
Asif | Biryani, Tea | Rs.1270.0
-----
Total Bill: Rs.5080
```

This image shows both saved bills custom and equal split.

Final Input and Output Image of both custom calculated spit and equal calculated split:

```
--- Available Menu ---
Item      | Price (Rs)
-----
Burger    | 450
Pizza     | 2200
Tea       | 70
Sandwich  | 60
Pasta     | 200
Coffee    | 50
Biryani   | 300
Chicken   | 1500
-----

Enter number of members: 4

Enter name of member 1: Hamza
Enter food item for Hamza (or type 'no' to finish): Biryani
Enter food item for Hamza (or type 'no' to finish): sandwich
Enter food item for Hamza (or type 'no' to finish): no

Enter name of member 2: Luqman
Enter food item for Luqman (or type 'no' to finish): Pizza
Enter food item for Luqman (or type 'no' to finish): burger
Enter food item for Luqman (or type 'no' to finish): no

Enter name of member 3: Hassan
Enter food item for Hassan (or type 'no' to finish): chicken
Warning: chicken not found in menu, try again.
Enter food item for Hassan (or type 'no' to finish): chicken
Enter food item for Hassan (or type 'no' to finish): pasta
Enter food item for Hassan (or type 'no' to finish): no

Enter name of member 4: Asif
Enter food item for Asif (or type 'no' to finish): biryani
Enter food item for Asif (or type 'no' to finish): tea
Enter food item for Asif (or type 'no' to finish): no

Choose split mode (custom/equal): custom

--- Final Bill Report ---
Timestamp: 2026-06-10 18:28:07
Name      | Items Ordered      | Amount
-----
Hamza     | Biryani, Sandwich  | Rs.360
Luqman    | Pizza, Burger      | Rs.2650
Hassan    | Chicken, Pasta     | Rs.1700
Asif      | Biryani, Tea       | Rs.370
-----
Total Bill: Rs.5080

Bill report appended to 'bill.txt'
```

This image of same persons and orders with custom split.

```

--- Available Menu ---
Item          | Price (Rs)
-----
Burger        | 450
Pizza         | 2200
Tea           | 70
Sandwich      | 60
Pasta         | 200
Coffee        | 50
Biryani       | 300
Chicken       | 1500
-----

Enter number of members: 4

Enter name of member 1: Hamza
Enter food item for Hamza (or type 'no' to finish): biryani
Enter food item for Hamza (or type 'no' to finish): sandwich
Enter food item for Hamza (or type 'no' to finish): no

Enter name of member 2: Luqman
Enter food item for Luqman (or type 'no' to finish): pizza
Enter food item for Luqman (or type 'no' to finish): burger
Enter food item for Luqman (or type 'no' to finish): no

Enter name of member 3: Hassan
Enter food item for Hassan (or type 'no' to finish): chicken
Enter food item for Hassan (or type 'no' to finish): pasta
Enter food item for Hassan (or type 'no' to finish): no

Enter name of member 4: Asif
Enter food item for Asif (or type 'no' to finish): biryani
Enter food item for Asif (or type 'no' to finish): tea
Enter food item for Asif (or type 'no' to finish): no

Choose split mode (custom/equal): equal

--- Final Bill Report ---
Timestamp: 2026-06-10 18:30:35
Name          | Items Ordered          | Amount
-----
Hamza         | Biryani, Sandwich     | Rs.1270.0
Luqman        | Pizza, Burger         | Rs.1270.0
Hassan        | Chicken, Pasta        | Rs.1270.0
Asif          | Biryani, Tea          | Rs.1270.0
-----
Total Bill: Rs.5080

Bill report appended to 'bill.txt'
(kivy_env) PS E:\Hamza projects\OOP Project>

```

This image is of same persons and order but with equal split.

Summary

- *Day Three achieved the final **40% completion**.*
- *Modules imported properly (*Person, FoodItem, Order, BillSplitter*).*
- *Menu loads from *menu.txt*.*
- *Orders saved into *bill.txt*.*
- *User input integrated for multiple customers.*
- *Bill splitting logic finalized.*

Final Report:

At least I have completed Cfeteria Ordering system Project which is based on OOP five python files are used and now this is simple project gui is not giving yet and Inshallah I am trying to give a unique GUI This is my final second semester Project Documentation. Source code of all explained files are given below.

SOURCE CODES:

Person.py.

```
class Person:

    def __init__(self, name):

        self.name = name

        self._ordered_items = []

    def add_item(self, food_item):

        self._ordered_items.append(food_item)

    def get_items(self):

        return self._ordered_items

    def __str__(self):

        return f"Person: {self.name}, Orders: {self._ordered_items}"
```

Food_Item.py.

```
class FoodItem:

    def __init__(self, item_name, price):

        self._item_name = item_name

        self._price = price if price > 0 else 0

    def get_name(self):

        return self._item_name

    def get_price(self):

        return self._price

    def set_price(self, new_price):

        if new_price > 0:

            self._price = new_price

        else:

            raise ValueError("Price must be
greater than zero")

    def __str__(self):
```

```
        return f"{self._item_name}:  
Rs.{self._price}"
```

Order.py.

```
class Order:  
  
    def __init__(self):  
        self._orders = {}  
  
    def add_order(self, person, food_item):  
        person_name = person.name if  
hasattr(person, "name") else person  
        if person_name not in self._orders:  
            self._orders[person_name] = []  
  
        self._orders[person_name].append(food_item)  
  
    def calculate_total(self):  
        total = 0  
        for items in self._orders.values():  
            for item in items:  
                total += item.get_price()
```

```
        return total

    def get_orders(self):
        return self._orders

    def __str__(self):
        return f"Orders: { {person:
[item.get_name() for item in items] for person,
items in self._orders.items()} }"
```

Bill_Splitter.py.

```
class BillSplitter:

    def __init__(self, orders):

        self.orders = orders    # dictionary:
        person -> list of ordered items

        self._total = None

    def split_custom(self, menu):

        individual_totals = {}
```

```
        for person, items in
self.orders.items():

            total = 0

            for item in items:

                if hasattr(item, "get_price"):

                    total += item.get_price()

                elif item in menu:

                    total += menu[item]

                else:

                    print(f"Warning: {item} not
found in menu")

            individual_totals[person] = total

        self._total =
sum(individual_totals.values())

        return individual_totals


def split_equal(self):

    if not self.orders:

        return {}

    if self._total is None:

        raise ValueError("Total not
calculated yet. Run split_custom first.")
```

```
        share = self._total / len(self.orders)

        return {person: share for person in
self.orders}

def get_total(self):

    return self._total

def set_total(self, new_total):

    if new_total > 0:

        self._total = new_total

    else:

        raise ValueError("Total must be
greater than zero")

def choose_split(self, menu, mode="custom"):

    if mode == "custom":

        return self.split_custom(menu)

    elif mode == "equal":

        if self._total is None:

            self.split_custom(menu)

        return self.split_equal()
```

```
        else:
            raise ValueError("Invalid mode.
Choose 'custom' or 'equal'.")
```

Main.py.

```
import datetime
from Person import Person
from Food_Item import FoodItem
from Order import Order
from Bill_Splitter import BillSplitter

# --- Load Menu from File ---
def load_menu():
    menu = {}
    with open("menu.txt", "r") as f:
        for line in f:
            parts = line.strip().split(",")
            if len(parts) != 2:
                continue
```



```
        item, price = parts
        menu[item] = int(price)

    return menu

menu = load_menu()

# --- Show Menu ---

def show_menu():
    print("\n--- Available Menu ---")
    print(f"{'Item':<12} | {'Price (Rs) ':<10}")
    print("-" * 25)
    for item, price in menu.items():
        print(f"{item:<12} | {price:<10}")
    print("-" * 25)

show_menu()

# --- User Orders ---

num_members = int(input("\nEnter number of
members: "))

persons = []
```

```
orders = Order()

for i in range(num_members):
    name = input(f"\nEnter name of member {i+1}: ")
    person = Person(name)

    while True:
        item = input(f"Enter food item for {name} (or type 'no' to finish): ")
        if item.lower() == "no":
            break

        normalized_item = None
        for key in menu.keys():
            if key.lower() == item.lower():
                normalized_item = key
                break

        if normalized_item is None:
```

```
        print(f"Warning: {item} not found in  
menu, try again.")
```

```
        continue
```

```
        food_item = FoodItem(normalized_item,  
menu[normalized_item])
```

```
        person.add_item(food_item)
```

```
        orders.add_order(person, food_item)
```

```
    persons.append(person)
```

```
# --- Bill Calculation ---
```

```
total = orders.calculate_total()
```

```
bill_splitter = BillSplitter({
```

```
    person.name: [item.get_name() for item in  
orders.get_orders()[person.name]]
```

```
    for person in persons
```

```
})
```

```
mode = input("\nChoose split mode  
(custom/equal): ").lower()
```

```
shares = bill_splitter.choose_split(menu, mode)
```

```
# --- Timestamp ---

timestamp =
datetime.datetime.now().strftime("%Y-%m-%d
%H:%M:%S")

# --- Print Report ---

print("\n--- Final Bill Report ---")

print(f"Timestamp: {timestamp}")

print(f"{'Name':<12} | {'Items Ordered':<20} |
{'Amount':<6}")

print("-" * 45)

report_lines = []

for person in persons:

    items = ", ".join([item.get_name() for item
in person.get_items()])

    amount = shares[person.name]

    line = f"{person.name:<12} | {items:<20} |
Rs.{amount:<6}"

    print(line)

    report_lines.append(line)

print("-" * 45)
```

```
print(f"Total Bill: Rs.{total}")

# --- Save Report to File ---

with open("bill.txt", "a") as f:

    f.write("\n--- New Order Session ---\n")

    f.write(f"Timestamp: {timestamp}\n")

    f.write(f"{'Name':<12} | {'Items  
Ordered':<20} | {'Amount':<6}\n")

    f.write("-" * 45 + "\n")

    for line in report_lines:

        f.write(line + "\n")

    f.write("-" * 45 + "\n")

    f.write(f"Total Bill: Rs.{total}\n")

print("\nBill report appended to 'bill.txt'")
```